

Salient Object Detection

End-to-End ML/DL Project

CNN model that takes an image and predicts a saliency mask for the most visually important object.

1000

ECSSD pairs

70/15/15

train / val / test

5

slides max

Prepared by Dren Rustemi



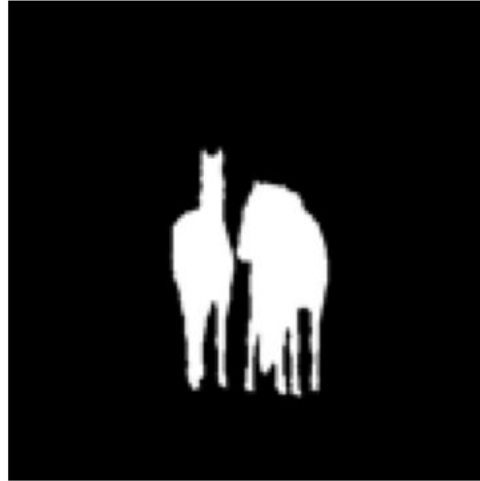
Dataset preparation & pipeline

From raw ECSSD images to normalized training tensors

Input image



Ground-truth mask



Input image + ground-truth saliency mask

- 1 Pair image files with their matching masks
- 2 Resize to 128×128 and normalize pixel values to 0–1
- 3 Apply basic augmentation during training
- 4 Final split: 700 train · 150 validation · 150 test

Requirement matched: public SOD dataset, preprocessing, augmentation, and 70/15/15 split.

Model design & improvements

U-Net style CNN built from scratch in TensorFlow/Keras

Architecture path

- 1 Encoder** extracts visual features
- 2 Bottleneck** learns compact representation
- 3 Decoder** rebuilds mask with skip connections
- 4 Sigmoid output** produces 1-channel saliency mask

Training

Adam optimizer · BCE + Dice loss · IoU / Precision / Recall / F1 metrics

Improvement strategy

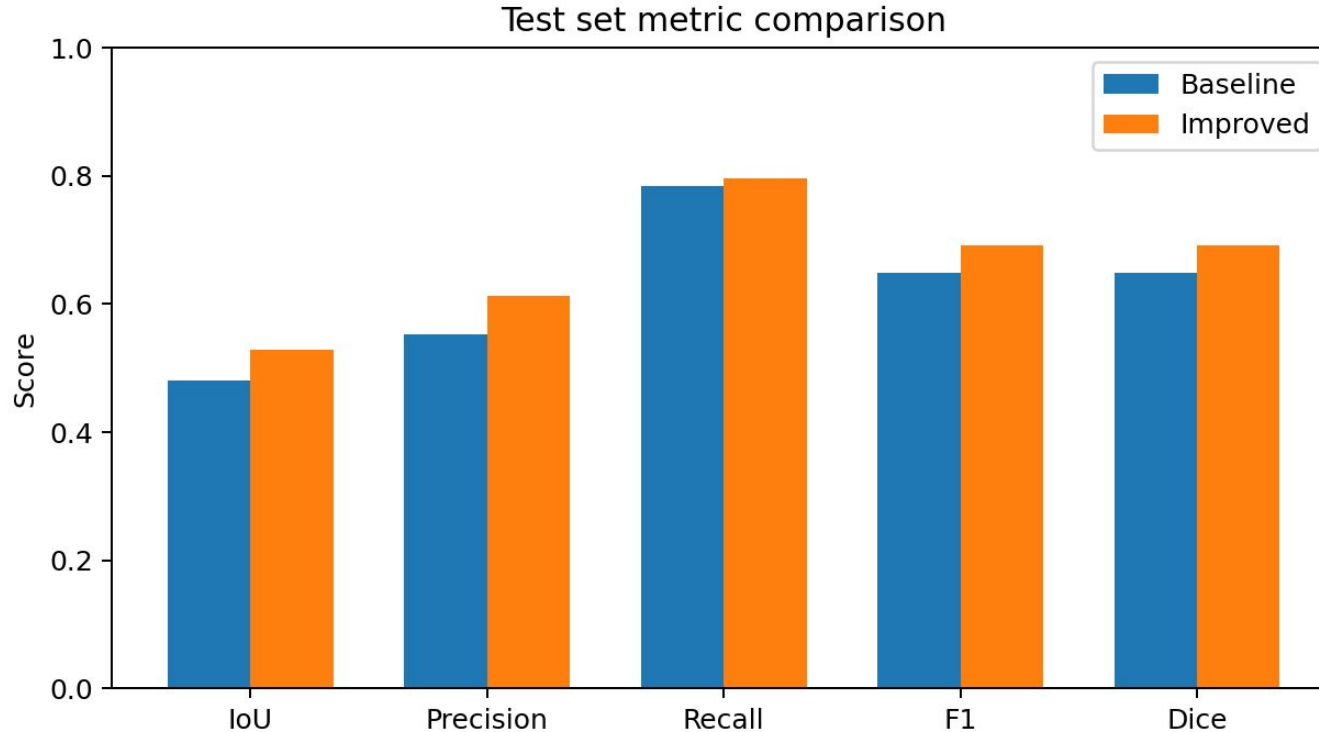
- 01 More filters**
base_filters changed from 8 to 32
- 02 Dropout layer**
dropout = 0.05 to reduce overfitting

Why compare?

Baseline vs. improved results show whether the changes actually helped.

Evaluation results

The improved model achieved better segmentation quality on the test set



0.529

Improved IoU

0.692

Improved F1/Dice

+10.2%

relative IoU gain

Main interpretation

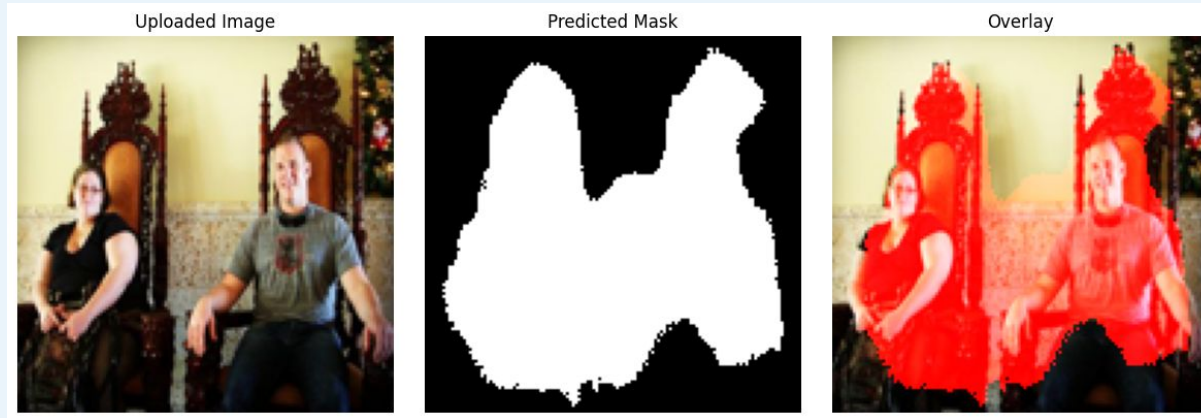
Precision improved from 0.553 to 0.612.

Recall improved from 0.785 to 0.797.

The improved model is slower, but still fast enough for demo use.

Demo & final conclusion

Live workflow: upload image → predict mask → display overlay

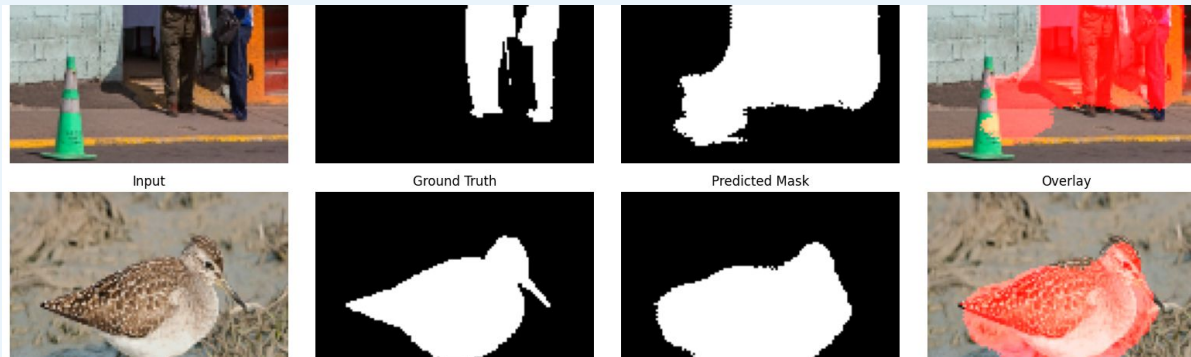


Demo output

Upload a new image inside the notebook.
The image is resized and normalized automatically.

The improved U-Net predicts a saliency mask.
The notebook shows the predicted mask and overlay.

Example inference time: 0.0951 seconds.



Final choice: improved U-Net

It produced stronger IoU, Precision, Recall, and F1/Dice results while meeting the project requirements for a working SOD demo.